



Interactive Digital Art and Design SE608

Proof of Concept - Concept Alignment Year IV

Student Name:	<i>Vit Prechtl</i>	Student Number:	<i>C00298335</i>
Working Title:	<i>Procedural Building Generator</i>		
PoC Date:	<i>[DD/MM/YYYY]</i>	PoC Version:	<i>Version 1</i>
How to use this document:	<i>Replace the grey guidance text with project-specific content. This Proof of Concept is intended to show whether the proposed FYP idea is coherent and achievable. The document should demonstrate a clear link between the project theme, objectives, user needs, features, tools/technologies, implementation approach, expected output and evaluation. Keep the scope focused on proving the concept rather than developing the complete final project.</i>		

1. Project Theme / Problem:	<i>[Explain the central theme, problem or opportunity that the FYP addresses. What is the project really about? Why is this theme important or interesting? Keep the explanation focused and understandable to someone outside the project area.]</i> This project addresses a couple of problems within environment 3D content production. Specifically, the time and repetitive manual effort required to build various architecture buildings and cityscapes by hand in Blender. My approach accelerates the process and, in some cases, avoids the problem of repetitive manual work entirely compared to the traditional workflows when it comes to creating cities. Without these tools and like the one I am on my journey to develop, the whole process would have of creating environments would need to follow a very traditional workflow. A traditional workflow of creating a single detailed building would require manually 3D modelling, Unwrapping, Texturing, Optimising, and Placing every wall, window, door, floor, roof, and other objects that create tertiary detail on the whole building. Then that process would need to be repeated for every NEW structure. Most of the time, when creating a single hero asset this is manageable, but when the aim is to populate entire environments which are needed in game environments, architecture visualization, and films.. This process becomes extremely time consuming, and very difficult to iterate on. So if a Director or a Client wants to change a building's height, style, amount of windows, different roof tiles, or a layout, the artist may have to redo large parts of the model from scratch and in some cases, do the entire model from scratch especially when it comes to changing the layout. Procedural generation offers a great solution to this problem. Instead of modelling each building by hand, a procedural system could utilize modular assets (kinda like lego bricks) and then lay them and control them (Creating a structure from the lego bricks) based on system defined rules, parameters, and algorithms to generate a building automatically. An Artist input would then be required to get the desired result by defining variables such as number of floors, footprint shape, window spacing, or roof type. This way, an Artist can generate many unique variations quickly, and adjust them instantly by changing parameters, rather than remodeling. I consider this theme important because it sits perfectly at an intersection of Art and technical problem-solving. In the industry, this is becoming a core skill in CG and Games to scale up environment production without the cost of proportional manual effort. So this problem contains both the technical and creative aspect. Technically it is a great chance to explore procedural modeling techniques using Blender Geometry nodes and Python Scripting, and creatively great in aid of faster iteration and greater variety in environment design, which is directly useful for a filmmaker, CG artist that needs to populate scenes effectively without sacrificing visual quality.
2. Project Aim:	<i>[State the overall aim in one clear sentence. The aim should describe the main outcome the project intends to achieve, not a list of technologies to be used.]</i> To develop a flexible and optimized procedural building generation tool in Blender that enables artists to quickly create varied, customizable architectural structures, and reduce the manual modelling efforts required to populate 3D environments, whether for use in game, film, or visualization environments.
3. Project Objectives:	<i>[List 3-5 specific objectives. Each objective should contribute directly to the project aim and should be achievable within the FYP timeframe. Avoid vague objectives such as "learn Unity" or "make a good game".]</i> 1. Design and implement a parametric building generation system in Blender using Geometry nodes. 2. Develop a procedural placement logic for modular 3D assets 3. Build a seed-based randomization system. 4. Evaluate the tool's effectiveness and polish accordingly (No user testing. Self-tested)

4. Target Users / Audience:	<i>[Identify who will use, experience or benefit from the project. Briefly describe their needs, expectations or context. If the project does not have a conventional end user, explain who the intended audience or stakeholder is.]</i> The primary target audiences are 3D artists, indie filmmakers, or environment designers working in games or films. So particularly those who need to populate scenes with buildings but lack time, budget, technical skill, or team size to model each structure by hand, or develop a procedural system. This could include solo developers, small studios, freelance CG artists, or indie filmmakers. Most of the time, this target audience doesn't have many resources to burn and has a lot of other things they need to focus on. These users typically need an accessible tool that will help them generate a large volume of visually varied assets quickly, without compromising too much on visual quality or control. This tool is heavily focused on the ease of use, since many potential users are artists first and second technicians. These people want a tool that fits into an existing Blender workflow, in which they would want to change and that which would provide a meaningful creative control such as the adjustment of style of the building, layout, building scale, and other necessary details and properties. Most importantly, removing the need for extensive scripting knowledge to operate, and repetitive, slow manual labor of modelling walls, windows, and roofs one at the time. I would love to make this project on a level of commercial product, but since this is a Final Year Project and even if project will be close to a same level of standard, it will probably end up as a demonstration of my technical and creative competency, and the observation skills to identify real production problems and design a tool-based solution to address it.
5. Main User Requirements / Use Cases:	<i>[Describe the important things a user must be able to do. Use short use cases or a simple user journey. Example structure: User -> Action -> System Response -> Outcome. Focus only on interactions that are central to the project objectives.]</i> Use case 1 – Generating a building User: An artist wants to quickly create a building without manual modeling. Action: User adds the procedural building generator via the side panel using the append button. The user will then be exposed with basic parameters such as the width and height of a building, and number of floors. System Response: The system automatically generates a complete building mesh. Outcome: Production ready to use building model, created in seconds instead of hours. The user will then be exposed with parameters such as the width and height of a building, and number of floors. Use case 2 – Adjusting building parameters User: Artist wants to change the number of floors in the building Action: User modifies the parameter for the number of floors and sets the index to 3. System Response: The system regenerates the geometry to have 2 additional floors in real time to reflect the new parameters, without requiring the user to remodel anything manually. Outcome: The user will have the requirement of 3 floors done in real time. The user can then continue iterating quickly on the design, comparing variations without starting over each time or spending time on technical slowdowns. Use case 3 – Creating variation across multiple buildings User: Artist needs to populate an environment with multiple distinct buildings rather than clones of one building model. Action: User duplicates the generator setup and adjusts parameters or uses a different seed for many of the inputs. The user then repeats this process for as many buildings as the user needs. System Response: The system produces visually distinct buildings from the same underlying system rules without any obvious repetition. Outcome: A quickly populated environment with architectural variety, with minimal manual duplication or effort.

6. Required Features / Functional Requirements:	<i>List the core features required to achieve the project objectives. Distinguish between features that are essential to prove the idea and features that are optional or only improve presentation.</i> Feature 1: Parametric Building Generation The system generates a complete building mesh from a small set of input parameters. Feature 2: Real Time / Near Real Time Updates Changing any parameter updates to the building without manual remodeling, with low or no delay. Feature 3: Randomization via seed A seed value to produce random variations of some rule sets, so users can generate multiple unique buildings fast. Feature 3: Side Panel User Interface A simple user interface so the user can easily make changes and work with the tool.
--	---

7. Proposed Tools and Technologies:	<i>Do not provide a simple software list. For every major tool or technology, explain what it will do in the project and which objective or feature it supports. Remove any tool that does not have a clear purpose.</i>			
	Tool / Technology	Purpose in Project	Feature / Objective Supported	Why Suitable?
	Blender	1. Model the 3D assets. 2. UV unwrap the 3D Assets. 3. Texture 3D assets using trim sheets. (Defined textures for flexible use) 4. Design and create the overall generation system.	Parametric Building Generation	Blender is a very versatile tool with a massive community that is constantly interested in improving Blender workflows as a whole. So, this tool could help even more people that are adapted to it. I chose Blender as it is a tool I am familiar with, has a bigger market than other 3D software, is very good for 3D modeling, unwrapping, and has a user-friendly environment for plugin creation.
	Adobe Substance Painter	Texture some of the 3D assets.	Modular 3D Assets for Building Generation	Substance Painter is an industry standard in texturing workflows. It has very good tools for generating masks, efficient layer based (photoshop like) workflow, real time control, and precision that will be necessary when creating unique scene props.
	Affinity	To create trim texture sheets, edit textures, and manipulate images.	Modular 3D Assets for Building Generation	Since Affinity got purchased by Canva, it gained massive attention. Since it combines what Adobe provides as multiple apps into a single package for free. Apart from that Affinity as way more stable, fast, and snappy to use. As a creator I can use the vector tools that I would find in Adobe Illustrator, seamlessly with pixel tools that I would find in Photoshop. It also has many other layouts and features that are suitable for manipulating photos that would be found in Adobe Lightroom. In short, I feel that affinity is the best choice because of its seamless workflow and accessibility, and I can see it soon being adapted by major industries.
	Visual Studio Code	Writing the overall plugin interface using Python, that easily and seamlessly works with the created geometry nodes.	Simple user interface	It is way more responsive and effective to use than Blender's built-in text editor. Blender text editor cannot use short-cuts when navigating and working with code, therefore it is slower to code and harder to debug. Blender's text editor is basically as capable as notepad.
[Type here]	[Type here]	[Type here]	[Type here]	
8. Project Alignment Matrix:	<i>This is the key section. Show the logical connection from each project objective to a required feature, the technology used to deliver it, how the PoC will provide evidence, and what the final project should produce.</i>			
Project Objective	Required Feature	Tool / Technology	PoC Evidence / Test	Expected Final Outcome

	<table><tr><td>Design and implement a parametric building generation system that constructs walls, floors, and roofs from user-defined inputs.</td><td>Parametric building generation with adjustable dimensions</td><td>Blender Geometry Nodes</td><td>A single Geometry Nodes setup generates a basic white box-massing building (walls + floor slabs) when footprint, floor count, and height is changed.</td><td>A working parametric building generator node group that reliably produces correct geometry across a defined input range.</td></tr><tr><td>Develop procedural placement logic for facade elements (windows and doors) that adapts to varying building dimensions</td><td>Procedural openings for windows, doors, etc.</td><td>Blender Geometry Nodes</td><td>A test building at 2-3 different sizes shows windows/doors placed.</td><td>A facade generation module that scales correctly across a range of footprint shapes/sizes, and floor counts, integrated into the main generator.</td></tr><tr><td>Build a seed-based randomization system producing multiple visually distinct buildings from the same rule set.</td><td>Randomization via seed.</td><td>Blender Geometry Nodes</td><td>A line of 5 buildings generated from one seed-driven setup, visually compared to confirm none of them look identical</td><td>A seed parameter integrated into the main tool, allowing rapid generation of unique building variants at a scene scale.</td></tr></table>	Design and implement a parametric building generation system that constructs walls, floors, and roofs from user-defined inputs.	Parametric building generation with adjustable dimensions	Blender Geometry Nodes	A single Geometry Nodes setup generates a basic white box-massing building (walls + floor slabs) when footprint, floor count, and height is changed.	A working parametric building generator node group that reliably produces correct geometry across a defined input range.	Develop procedural placement logic for facade elements (windows and doors) that adapts to varying building dimensions	Procedural openings for windows, doors, etc.	Blender Geometry Nodes	A test building at 2-3 different sizes shows windows/doors placed.	A facade generation module that scales correctly across a range of footprint shapes/sizes, and floor counts, integrated into the main generator.	Build a seed-based randomization system producing multiple visually distinct buildings from the same rule set.	Randomization via seed.	Blender Geometry Nodes	A line of 5 buildings generated from one seed-driven setup, visually compared to confirm none of them look identical	A seed parameter integrated into the main tool, allowing rapid generation of unique building variants at a scene scale.
Design and implement a parametric building generation system that constructs walls, floors, and roofs from user-defined inputs.	Parametric building generation with adjustable dimensions	Blender Geometry Nodes	A single Geometry Nodes setup generates a basic white box-massing building (walls + floor slabs) when footprint, floor count, and height is changed.	A working parametric building generator node group that reliably produces correct geometry across a defined input range.												
Develop procedural placement logic for facade elements (windows and doors) that adapts to varying building dimensions	Procedural openings for windows, doors, etc.	Blender Geometry Nodes	A test building at 2-3 different sizes shows windows/doors placed.	A facade generation module that scales correctly across a range of footprint shapes/sizes, and floor counts, integrated into the main generator.												
Build a seed-based randomization system producing multiple visually distinct buildings from the same rule set.	Randomization via seed.	Blender Geometry Nodes	A line of 5 buildings generated from one seed-driven setup, visually compared to confirm none of them look identical	A seed parameter integrated into the main tool, allowing rapid generation of unique building variants at a scene scale.												
9. PoC Demonstration / Evidence:	Describe the small demonstration, mock-up, workflow, prototype, experiment or test that will be used to show the idea is workable. The PoC does not need to be the final product. It should test the most important or uncertain part of the concept. Include screenshots, diagrams, wireframes, short results or a link to evidence where appropriate. Process Flow Diagram															
10. Technology Suitability / Alternatives:	Where the technology choice is important, compare the proposed approach with at least one realistic alternative. The final choice should be based on project requirements, not familiarity alone. <table><tr><th>Requirement</th><th>Proposed Approach / Tool</th><th>Alternative Considered</th><th>Reason for Final Choice</th></tr><tr><td>Procedural capability</td><td>Blender Geometry Nodes</td><td>SideFX Houdini</td><td>Blender uses the same node-based procedural paradigm as Houdini. It provides all the node types needed for this project’s scope: mesh generation, instancing, and attribute-driven variation. Houdini, on the other hand, offers a broader procedural toolset, most of which is unnecessary overhead for building generators of this scope.</td></tr></table>	Requirement	Proposed Approach / Tool	Alternative Considered	Reason for Final Choice	Procedural capability	Blender Geometry Nodes	SideFX Houdini	Blender uses the same node-based procedural paradigm as Houdini. It provides all the node types needed for this project’s scope: mesh generation, instancing, and attribute-driven variation. Houdini, on the other hand, offers a broader procedural toolset, most of which is unnecessary overhead for building generators of this scope.							
Requirement	Proposed Approach / Tool	Alternative Considered	Reason for Final Choice													
Procedural capability	Blender Geometry Nodes	SideFX Houdini	Blender uses the same node-based procedural paradigm as Houdini. It provides all the node types needed for this project’s scope: mesh generation, instancing, and attribute-driven variation. Houdini, on the other hand, offers a broader procedural toolset, most of which is unnecessary overhead for building generators of this scope.													

	<table><tr><td>Integration</td><td>Geometry Nodes</td><td>SideFX Houdini</td><td>Native to Blender – no export/import step needed between modeling, generation, and rendering.</td></tr><tr><td>User accessibility</td><td>Python + Geometry Nodes</td><td>Geometry Nodes Only</td><td>Final tool interface will expose a cleaner simplified panel for non-technical users.</td></tr><tr><td>Feasibility for FYP timeframe</td><td>Blender</td><td>SideFX Houdini</td><td>I am already familiar with Blender, plus the community resources are extensive for problem solving. Houdini would have a steeper learning curve, and my unfamiliarity with the tool would let me run into unexpected issues, solving them would likely not fit within the project's time frame. (Licensing setup alone would be complicated) So in that time frame, I am more likely to develop a stronger project using Blender.</td></tr></table>	Integration	Geometry Nodes	SideFX Houdini	Native to Blender – no export/import step needed between modeling, generation, and rendering.	User accessibility	Python + Geometry Nodes	Geometry Nodes Only	Final tool interface will expose a cleaner simplified panel for non-technical users.	Feasibility for FYP timeframe	Blender	SideFX Houdini	I am already familiar with Blender, plus the community resources are extensive for problem solving. Houdini would have a steeper learning curve, and my unfamiliarity with the tool would let me run into unexpected issues, solving them would likely not fit within the project's time frame. (Licensing setup alone would be complicated) So in that time frame, I am more likely to develop a stronger project using Blender.																			
Integration	Geometry Nodes	SideFX Houdini	Native to Blender – no export/import step needed between modeling, generation, and rendering.																													
User accessibility	Python + Geometry Nodes	Geometry Nodes Only	Final tool interface will expose a cleaner simplified panel for non-technical users.																													
Feasibility for FYP timeframe	Blender	SideFX Houdini	I am already familiar with Blender, plus the community resources are extensive for problem solving. Houdini would have a steeper learning curve, and my unfamiliarity with the tool would let me run into unexpected issues, solving them would likely not fit within the project's time frame. (Licensing setup alone would be complicated) So in that time frame, I am more likely to develop a stronger project using Blender.																													
11. Scope - Core vs Optional Features:	Prioritise features as Must Have, Should Have or Could Have. The PoC should focus on the Must Have features and the areas with the greatest uncertainty or risk. <table><tr><th>Feature</th><th>Priority</th><th>Reason / Objective Supported</th><th>Include in PoC?</th></tr><tr><td>Parametric Building Generation</td><td>Must Have</td><td>It is a core building block and proof that a rule-based system can replace manual modeling.</td><td>Yes</td></tr><tr><td>Randomization via seed</td><td>Must Have</td><td>It is necessary to be able to create variations quickly or experiment with different looks, especially at a scene scale.</td><td>Yes</td></tr><tr><td>Simple User Interface</td><td>Could Have</td><td>Simple User Interface is the part that could make or break the usability of the tool. No matter how good the tool is, if it's not accessible to any technical level and easy to use, artists just won't use it.</td><td>Yes</td></tr><tr><td>Real-Time / Near Real-Time Updates</td><td>Must Have</td><td>It is a must have because the tool should save time and give the artist a tangible access to the creative process, allowing them to see the result of a parameter change immediately, so they are motivated to continue making changes.</td><td>Yes</td></tr><tr><td>Performance Optimization for Viewport (Proxy generation)</td><td>Should Have</td><td>Users should have access to a button to toggle proxies, to improve their viewport performance, especially when creating more complex buildings and environments.</td><td>Yes</td></tr><tr><td>Option to let Users use/replace assets with their own, based on the modular workflow.</td><td>Should Have</td><td>Users should be able to create their own modular assets and use them within the designed generation system, so they can experiment with random seeds, parameters, and generate various buildings</td><td>Yes</td></tr></table>				Feature	Priority	Reason / Objective Supported	Include in PoC?	Parametric Building Generation	Must Have	It is a core building block and proof that a rule-based system can replace manual modeling.	Yes	Randomization via seed	Must Have	It is necessary to be able to create variations quickly or experiment with different looks, especially at a scene scale.	Yes	Simple User Interface	Could Have	Simple User Interface is the part that could make or break the usability of the tool. No matter how good the tool is, if it's not accessible to any technical level and easy to use, artists just won't use it.	Yes	Real-Time / Near Real-Time Updates	Must Have	It is a must have because the tool should save time and give the artist a tangible access to the creative process, allowing them to see the result of a parameter change immediately, so they are motivated to continue making changes.	Yes	Performance Optimization for Viewport (Proxy generation)	Should Have	Users should have access to a button to toggle proxies, to improve their viewport performance, especially when creating more complex buildings and environments.	Yes	Option to let Users use/replace assets with their own, based on the modular workflow.	Should Have	Users should be able to create their own modular assets and use them within the designed generation system, so they can experiment with random seeds, parameters, and generate various buildings	Yes
Feature	Priority	Reason / Objective Supported	Include in PoC?																													
Parametric Building Generation	Must Have	It is a core building block and proof that a rule-based system can replace manual modeling.	Yes																													
Randomization via seed	Must Have	It is necessary to be able to create variations quickly or experiment with different looks, especially at a scene scale.	Yes																													
Simple User Interface	Could Have	Simple User Interface is the part that could make or break the usability of the tool. No matter how good the tool is, if it's not accessible to any technical level and easy to use, artists just won't use it.	Yes																													
Real-Time / Near Real-Time Updates	Must Have	It is a must have because the tool should save time and give the artist a tangible access to the creative process, allowing them to see the result of a parameter change immediately, so they are motivated to continue making changes.	Yes																													
Performance Optimization for Viewport (Proxy generation)	Should Have	Users should have access to a button to toggle proxies, to improve their viewport performance, especially when creating more complex buildings and environments.	Yes																													
Option to let Users use/replace assets with their own, based on the modular workflow.	Should Have	Users should be able to create their own modular assets and use them within the designed generation system, so they can experiment with random seeds, parameters, and generate various buildings	Yes																													

	<table><tr><td></td><td></td><td>in their own style, and use the tool more freely.</td><td></td></tr><tr><td>Extra Detail Elements</td><td>Could Have</td><td>This would be nice to have, because it would encourage the user to use the tool, since it can provide them with more resources to work with, and make their scenes more unique.</td><td>No</td></tr><tr><td>Style Presets</td><td>Could Have</td><td>The tool could have a list of architecture styles, so that the user can pick between more than just a simple building, an example could be a skyscraper, a street shop, detached house, Asian temple, gothic church, medieval castle, or a cyberpunk building.</td><td>No</td></tr><tr><td>Street or City Layout Tool</td><td>Could Have</td><td>When creating an environment, it would be nice to have a couple of tools or templates to quickly populate a scene. An example such a template could be:</td><td>No</td></tr></table>			in their own style, and use the tool more freely.		Extra Detail Elements	Could Have	This would be nice to have, because it would encourage the user to use the tool, since it can provide them with more resources to work with, and make their scenes more unique.	No	Style Presets	Could Have	The tool could have a list of architecture styles, so that the user can pick between more than just a simple building, an example could be a skyscraper, a street shop, detached house, Asian temple, gothic church, medieval castle, or a cyberpunk building.	No	Street or City Layout Tool	Could Have	When creating an environment, it would be nice to have a couple of tools or templates to quickly populate a scene. An example such a template could be:	No				
		in their own style, and use the tool more freely.																			
Extra Detail Elements	Could Have	This would be nice to have, because it would encourage the user to use the tool, since it can provide them with more resources to work with, and make their scenes more unique.	No																		
Style Presets	Could Have	The tool could have a list of architecture styles, so that the user can pick between more than just a simple building, an example could be a skyscraper, a street shop, detached house, Asian temple, gothic church, medieval castle, or a cyberpunk building.	No																		
Street or City Layout Tool	Could Have	When creating an environment, it would be nice to have a couple of tools or templates to quickly populate a scene. An example such a template could be:	No																		
12. Feasibility Assessment:	Assess whether the project is realistically achievable. Consider: Technical - can the required features be built? Time - can a meaningful version be completed? Skills - can the required skills be developed in time? Resources - are software, hardware, data and participants available? Evaluation - can success be demonstrated or measured? <table><tr><th>Area</th><th>Question to Consider</th><th>Assessment / Evidence</th></tr><tr><td>Technical Feasibility</td><td>Can the required features be built?</td><td>Blender's Geometry Nodes system already supports parametric mesh generation, instancing, and attribute-driven randomization.</td></tr><tr><td>Time Feasibility</td><td>Can a meaningful version be completed in time?</td><td>The must have features form a complete, demonstrable system on their own, giving an achievable "minimum complete version" well before the deadline, with should/could have features as buffer work.</td></tr><tr><td>Skills Feasibility</td><td>Can the required skills be developed in time?</td><td>I already have a relevant Blender experience, reducing ramp-up time, and the main skill gap (advanced conditional geometry logic) is addressed by prioritising those features early.</td></tr><tr><td>Resource Feasibility</td><td>Are the necessary software, hardware and data available?</td><td>All required software is free and already installed, and no external participants, proprietary data or specialized hardware are needed, removing licensing, budget, and access risk.</td></tr><tr><td>Evaluation Feasibility</td><td>Can success be demonstrated or measured?</td><td>Success is measured through functional testing of valid geometry generation and a time/effort comparison against manual modeling, giving both qualitative and quantitative evidence for success.</td></tr></table>			Area	Question to Consider	Assessment / Evidence	Technical Feasibility	Can the required features be built?	Blender's Geometry Nodes system already supports parametric mesh generation, instancing, and attribute-driven randomization.	Time Feasibility	Can a meaningful version be completed in time?	The must have features form a complete, demonstrable system on their own, giving an achievable "minimum complete version" well before the deadline, with should/could have features as buffer work.	Skills Feasibility	Can the required skills be developed in time?	I already have a relevant Blender experience, reducing ramp-up time, and the main skill gap (advanced conditional geometry logic) is addressed by prioritising those features early.	Resource Feasibility	Are the necessary software, hardware and data available?	All required software is free and already installed, and no external participants, proprietary data or specialized hardware are needed, removing licensing, budget, and access risk.	Evaluation Feasibility	Can success be demonstrated or measured?	Success is measured through functional testing of valid geometry generation and a time/effort comparison against manual modeling, giving both qualitative and quantitative evidence for success.
Area	Question to Consider	Assessment / Evidence																			
Technical Feasibility	Can the required features be built?	Blender's Geometry Nodes system already supports parametric mesh generation, instancing, and attribute-driven randomization.																			
Time Feasibility	Can a meaningful version be completed in time?	The must have features form a complete, demonstrable system on their own, giving an achievable "minimum complete version" well before the deadline, with should/could have features as buffer work.																			
Skills Feasibility	Can the required skills be developed in time?	I already have a relevant Blender experience, reducing ramp-up time, and the main skill gap (advanced conditional geometry logic) is addressed by prioritising those features early.																			
Resource Feasibility	Are the necessary software, hardware and data available?	All required software is free and already installed, and no external participants, proprietary data or specialized hardware are needed, removing licensing, budget, and access risk.																			
Evaluation Feasibility	Can success be demonstrated or measured?	Success is measured through functional testing of valid geometry generation and a time/effort comparison against manual modeling, giving both qualitative and quantitative evidence for success.																			
13. Risks and Plan B:	Identify the main reasons the project might fail, be delayed or lose scope. Give a practical mitigation and an alternative approach. Include ethics approval, hardware dependency, unfamiliar technology or external participants where relevant. <table><tr><th>Risk / Uncertainty</th><th>What It Affects</th><th>Mitigation</th><th>Plan B / Alternative</th></tr></table>			Risk / Uncertainty	What It Affects	Mitigation	Plan B / Alternative														
Risk / Uncertainty	What It Affects	Mitigation	Plan B / Alternative																		

	<table border="1"> <tr> <td>Plugin Bugs</td><td>The user interface</td><td>Create a simple script from the beginning to test compatibility with Geometry nodes and possibly iterate on it over time.</td><td>Create the user interface in a geometry nodes modifier panel</td></tr> <tr> <td>Low Amount of Assets</td><td>Demonstration of the versatility and usefulness of the created system.</td><td>Make some assets using a projection mapping technique that eliminates the need for external texturing apps and saves time on modeling. Leaving detail of the models in the texture and making them more optimized.</td><td>Create assets in a simpler style but keep up a good artistic taste for a superior output.</td></tr> <tr> <td>Geometry Nodes Knowledge</td><td>The whole level of standard, and features in the tool.</td><td>Download and study different geometry nodes setups created by the community.</td><td><i>Buy a geometry nodes online video course to acquire skills faster</i></td></tr> <tr> <td>More delayed and no real-time feedback/updates</td><td>The effectiveness in saving time and motivation for the user to use the tool.</td><td>Make the system rely less on mesh generation and emphasize modular 3D asset instancing.</td><td>Make all the mesh using modular 3D assets and decimate them for proxies.</td></tr> </table>	Plugin Bugs	The user interface	Create a simple script from the beginning to test compatibility with Geometry nodes and possibly iterate on it over time.	Create the user interface in a geometry nodes modifier panel	Low Amount of Assets	Demonstration of the versatility and usefulness of the created system.	Make some assets using a projection mapping technique that eliminates the need for external texturing apps and saves time on modeling. Leaving detail of the models in the texture and making them more optimized.	Create assets in a simpler style but keep up a good artistic taste for a superior output.	Geometry Nodes Knowledge	The whole level of standard, and features in the tool.	Download and study different geometry nodes setups created by the community.	<i>Buy a geometry nodes online video course to acquire skills faster</i>	More delayed and no real-time feedback/updates	The effectiveness in saving time and motivation for the user to use the tool.	Make the system rely less on mesh generation and emphasize modular 3D asset instancing.	Make all the mesh using modular 3D assets and decimate them for proxies.
Plugin Bugs	The user interface	Create a simple script from the beginning to test compatibility with Geometry nodes and possibly iterate on it over time.	Create the user interface in a geometry nodes modifier panel														
Low Amount of Assets	Demonstration of the versatility and usefulness of the created system.	Make some assets using a projection mapping technique that eliminates the need for external texturing apps and saves time on modeling. Leaving detail of the models in the texture and making them more optimized.	Create assets in a simpler style but keep up a good artistic taste for a superior output.														
Geometry Nodes Knowledge	The whole level of standard, and features in the tool.	Download and study different geometry nodes setups created by the community.	<i>Buy a geometry nodes online video course to acquire skills faster</i>														
More delayed and no real-time feedback/updates	The effectiveness in saving time and motivation for the user to use the tool.	Make the system rely less on mesh generation and emphasize modular 3D asset instancing.	Make all the mesh using modular 3D assets and decimate them for proxies.														
14. PoC Outcome / Decision:	<i>Select one outcome and justify the decision below:</i> ☐ Proceed - the concept and proposed approach are suitable. ☐ Proceed with changes - the concept is feasible, but changes are required. ☐ Reconsider - the current concept or approach requires significant revision. <i>Justification: [Explain the decision in 3-5 sentences.]</i>																
15. Required Changes Before Full Development:	<i>List the specific changes that must be made as a result of the PoC. Examples: change technology, reduce scope, replace an input method, redefine a feature, modify the user flow, add a Plan B, or strengthen the technical component.</i>																
16. Next Steps:	<i>List the next 3-5 concrete development tasks in priority order. Each task should follow logically from the PoC decision and the revised project direction.</i>																
17. Supporting Material / References:	<i>Provide links or references to supporting research, technical documentation, comparable systems, design references, GitHub repository, screenshots, diagrams or short demonstration videos used to justify the PoC decisions.</i>																
Student declaration: The information in this Proof of Concept reflects the current proposed direction of the FYP and the evidence available at this stage. Signed: _____ Date: _____																	
For Office / Supervisor Use Only																	
PoC reviewed:	<i>[Supervisor notes]</i>																

Main issues / conditions:	<i>[Supervisor notes]</i>
Approved direction / required changes:	<i>[Supervisor notes]</i>
Supervisor:	<i>[Supervisor notes]</i>
Signed / Date:	<i>[Supervisor notes]</i>